Using a VPN to Bypass a Firewall

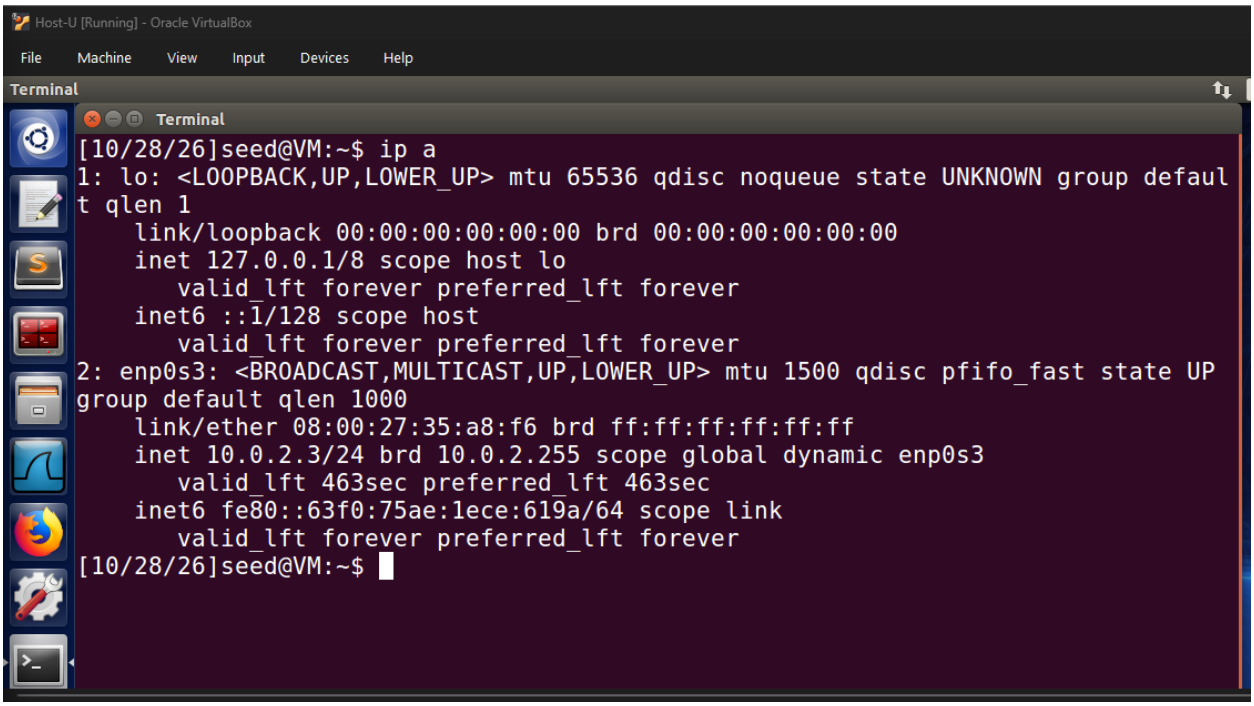
A firewall can block a website. A VPN can get around that block. In this lab, I blocked Facebook on one computer. Then I used a private tunnel (a VPN) to reach it anyway. The firewall stayed on the whole time.

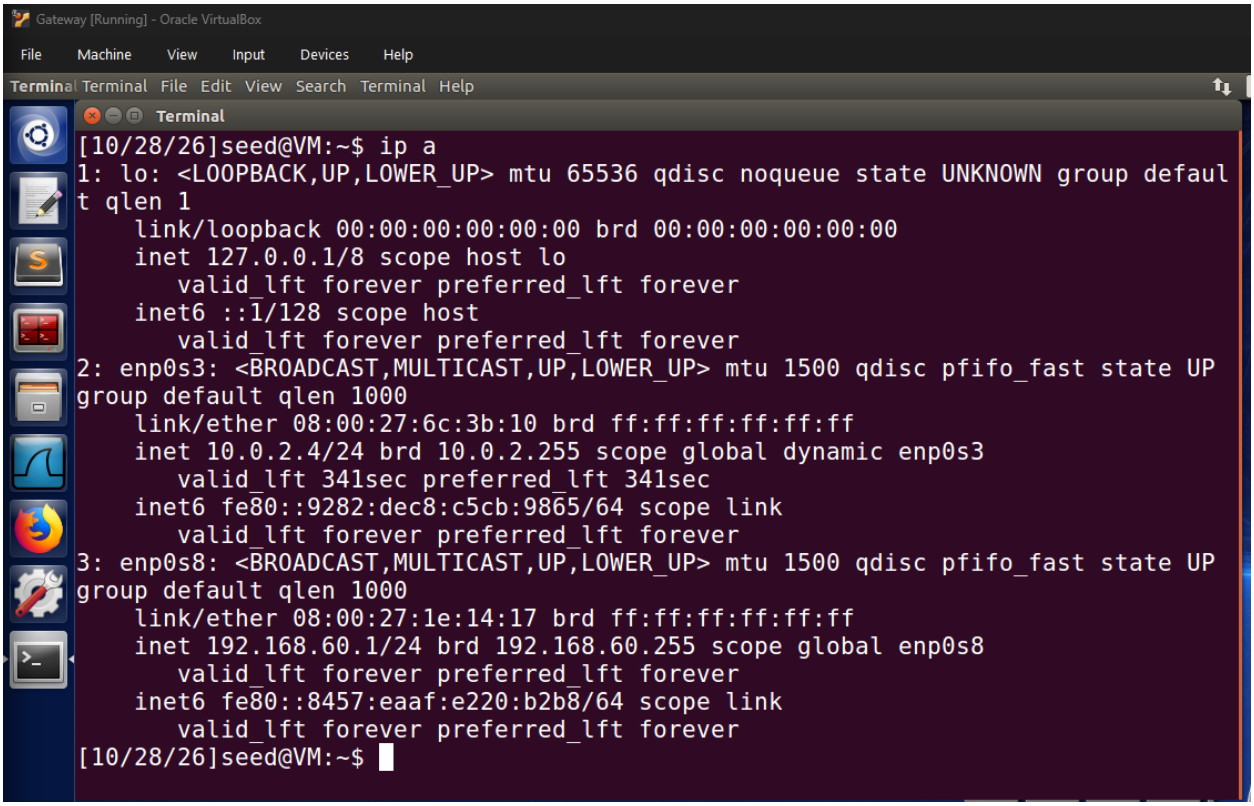
Two computers are used. Host U is the computer behind the firewall (address 10.0.2.3). The Gateway is a second computer outside the firewall (address 10.0.2.4). The Gateway helps Host U reach the blocked site.

TASK 1

Run ip a to confirm each hosts IP and to get the real interface name (`enp0s3`) for the firewall rule.

Goal: get the two computers ready. I ran the command ip a on each one. It shows the computer's address and the name of its network card. Both computers use a network card called enp0s3. I need that name for the firewall rule.

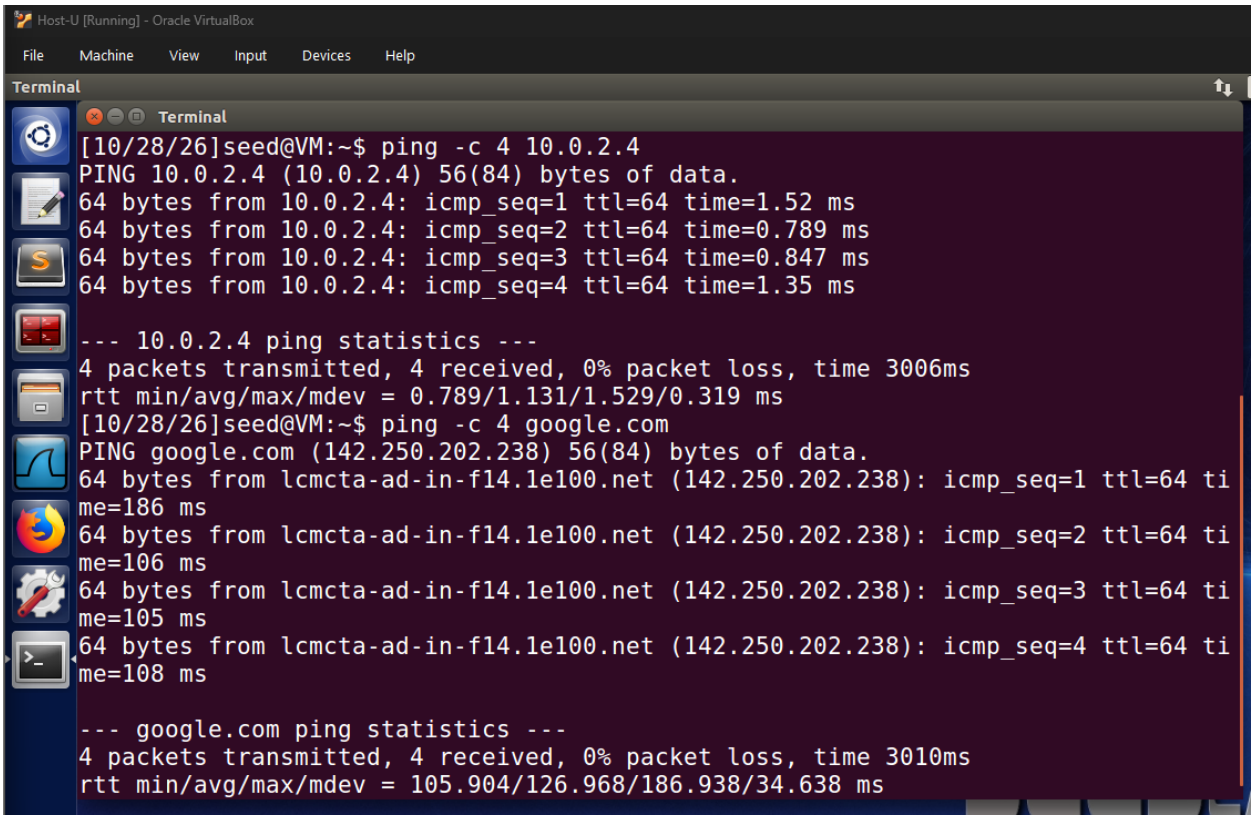




Ping Gateway and google.com

A ping sends a small test message and waits for an answer. Host U got an answer from the Gateway and from google.com every time, with no loss. This shows the network works before I add the firewall.

```
ping -c 4 10.0.2.4
ping -c 4 google.com
```

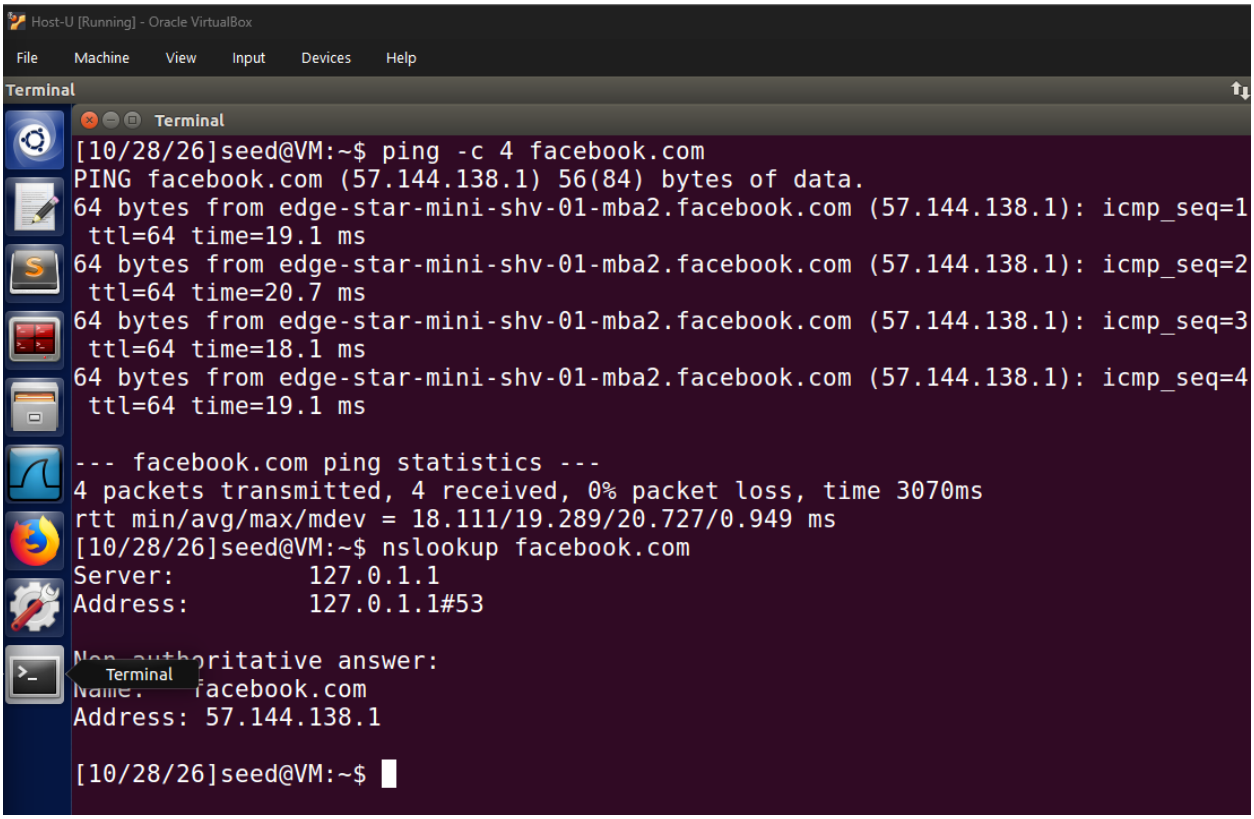


Task 2: Set up the firewall (on Host U)

Step 2.1

I chose facebook.com as the site to block. First I checked that it works. Host U gets answers from Facebook, and its address is 57.144.138.1.

```
ping -c 4 facebook.com
nslookup facebook.com
```

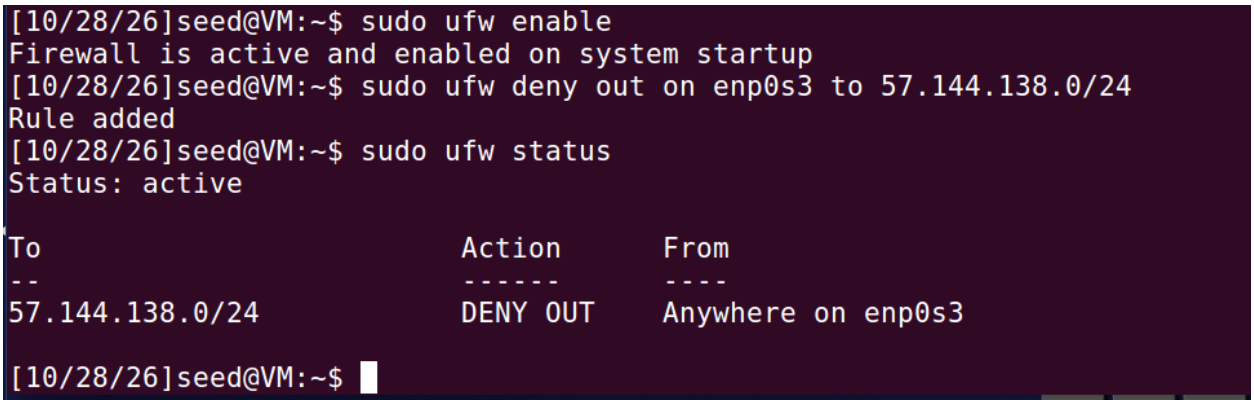


Step 2.2: Enable the firewall and block the site

I turned the firewall on at Host U and added one rule. The rule blocks everything sent to 57.144.138.0/24. That is Facebook's address and the addresses next to it. I put the rule on the real network card (enp0s3). I did not put it on the VPN's virtual card, so the VPN is not blocked.

```
sudo ufw enable
sudo ufw deny out on enp0s3 to 57.144.138.0/24
sudo ufw status
```

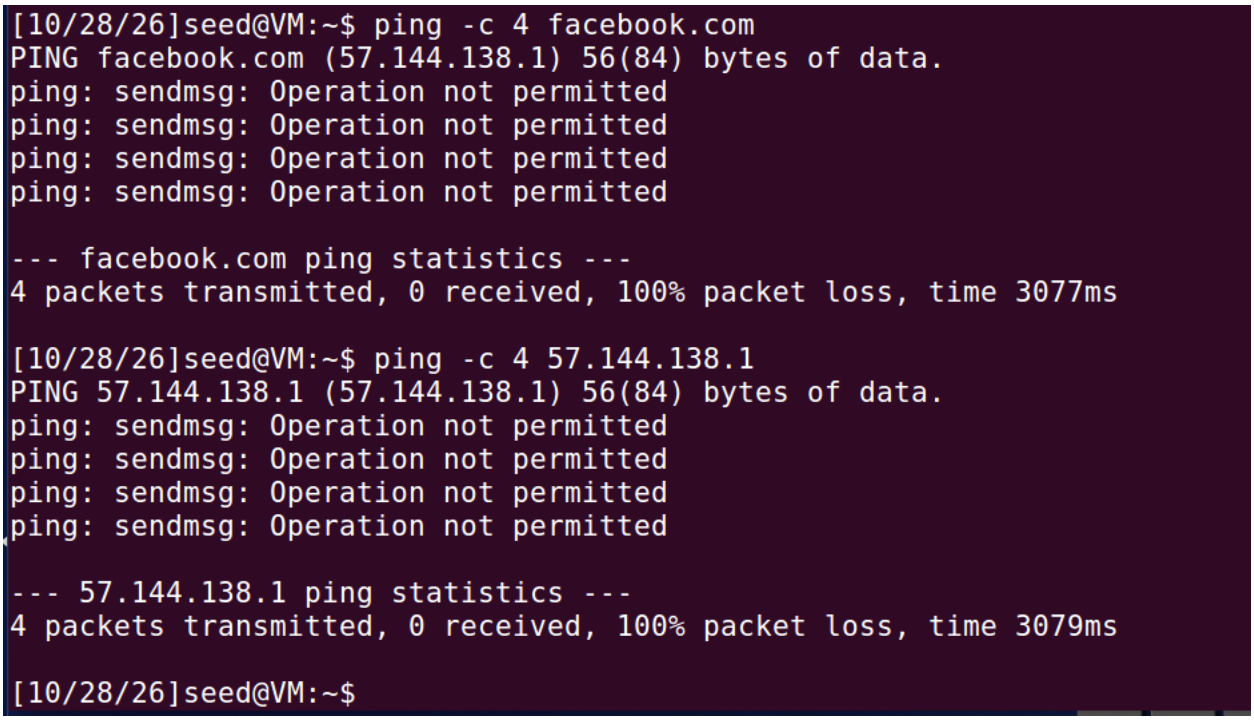
DENY OUT rule



Step 2.3: Show the block works

I tried to reach Facebook. Both tries failed, and every message was lost (100% loss). The firewall works.

```
ping -c 4 facebook.com
ping -c 4 57.144.138.1
```



TASK 3

The idea: Host U cannot send messages to Facebook directly, because the firewall stops them. So Host U sends them through a private tunnel to the Gateway. The Gateway is outside the firewall, so it can pass them on to Facebook. Facebook's answers come back the same way. The firewall only sees messages going to the Gateway. It never sees messages going to Facebook.

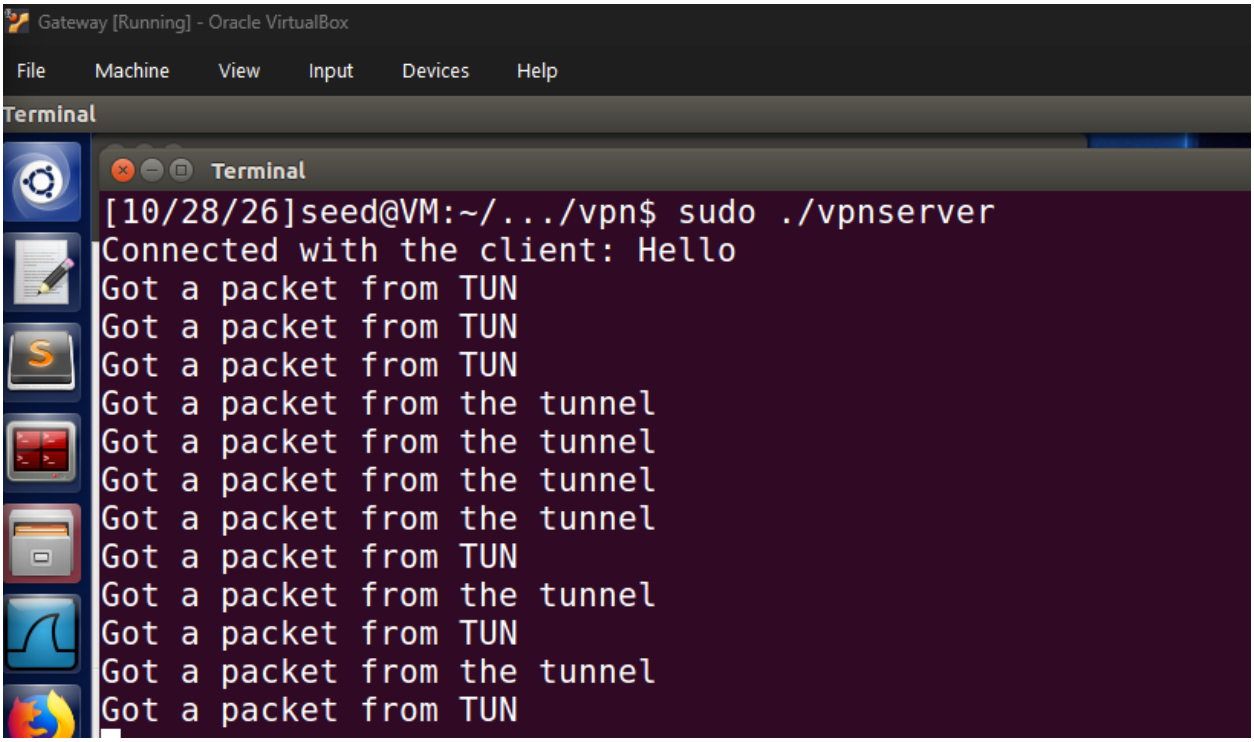
The tunnel is made with the two programs from the lab: vpnserver and vpnclient. This tunnel is not encrypted. It gets around the firewall, but it does not hide the content of the messages.

Step 1: Run the VPN server (on Gateway)

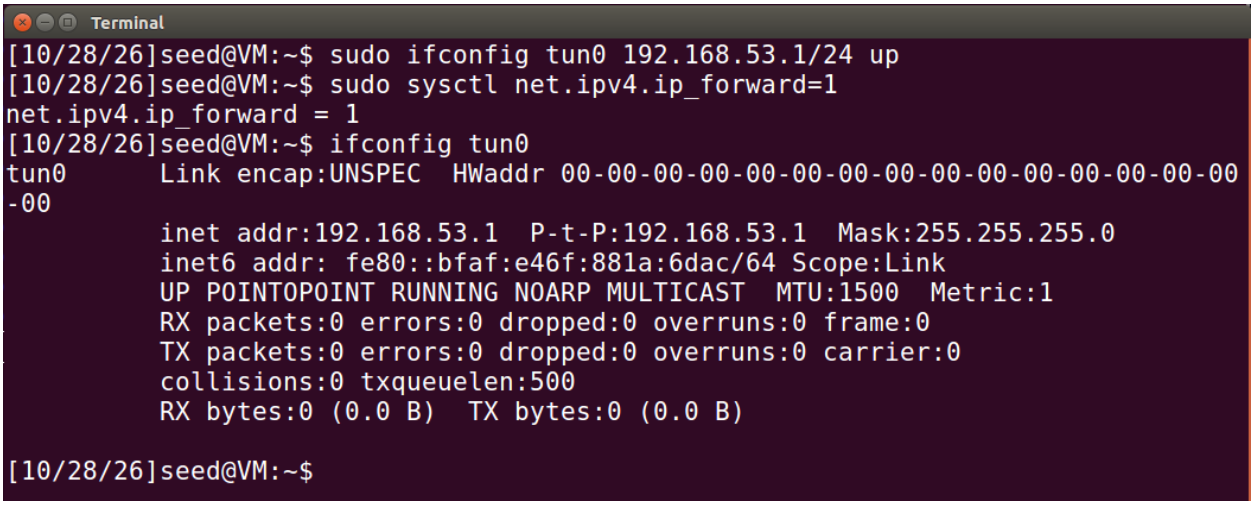
The server program runs on the Gateway and waits for Host U to connect. It creates a virtual network card called tun0. This is the Gateway's end of the tunnel. I gave tun0 the address 192.168.53.1. I also turned on IP forwarding. This lets the Gateway pass messages on to other places. Without it, the Gateway would keep every message for itself and could not pass Host U's traffic on to Facebook.

```
sudo ./vpnserver
sudo ifconfig tun0 192.168.53.1/24 up
sudo sysctl net.ipv4.ip_forward=1
```

Server running



Ifconfig tun0 output



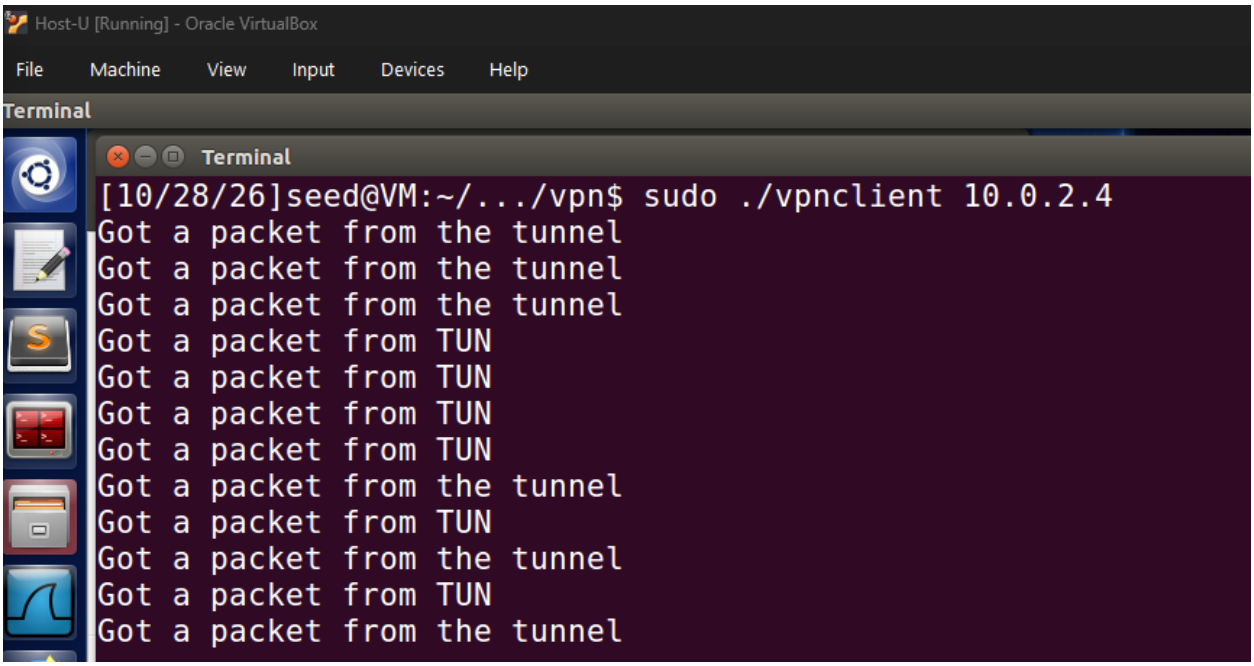
Step 2: Run the VPN client (Host U)

The client program runs on Host U and connects to the Gateway at 10.0.2.4. It creates its own tun0, which is Host U's end of the tunnel, with the address 192.168.53.5. To test the tunnel, I pinged the Gateway's tunnel address, 192.168.53.1. It answered, so the tunnel works.

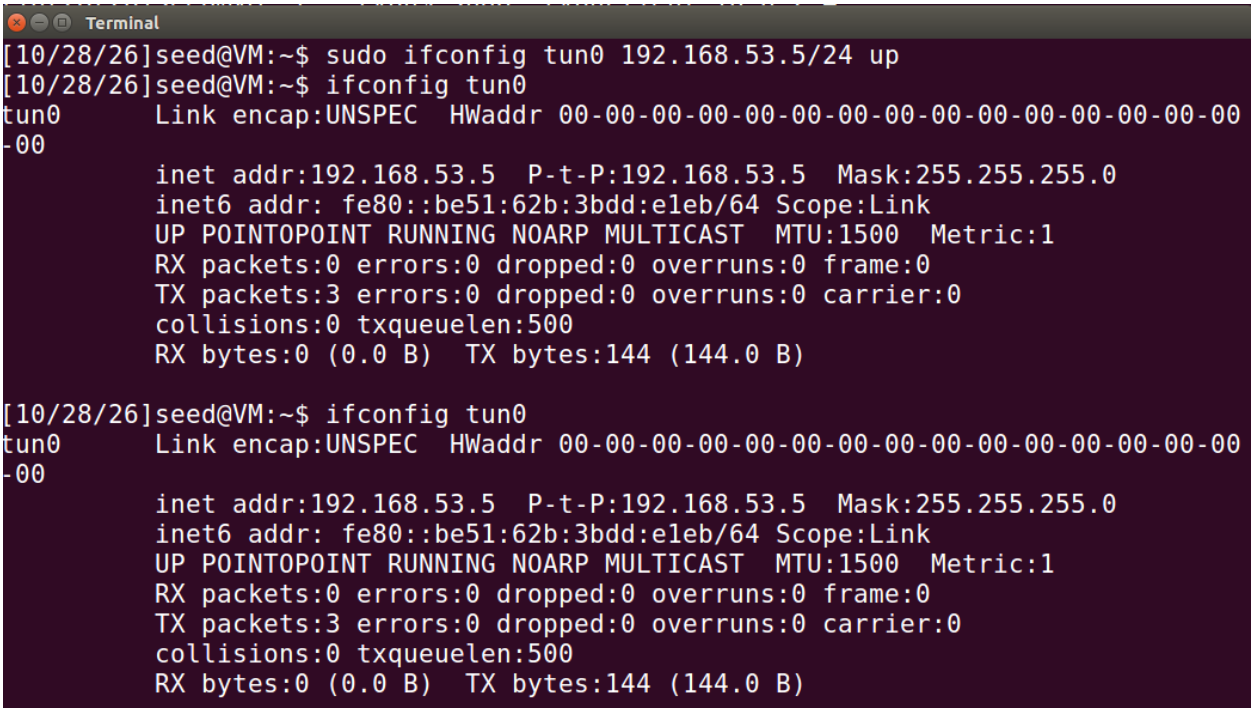
A problem I fixed: at first, this ping failed. I had typed the tunnel address without the /24 part, so the computer did not know to use the tunnel. I added /24, restarted the server, and then restarted the client. After that, it worked.

```
sudo ./vpnclient 10.0.2.4
sudo ifconfig tun0 192.168.53.5/24 up
ping -c 3 192.168.53.1
```

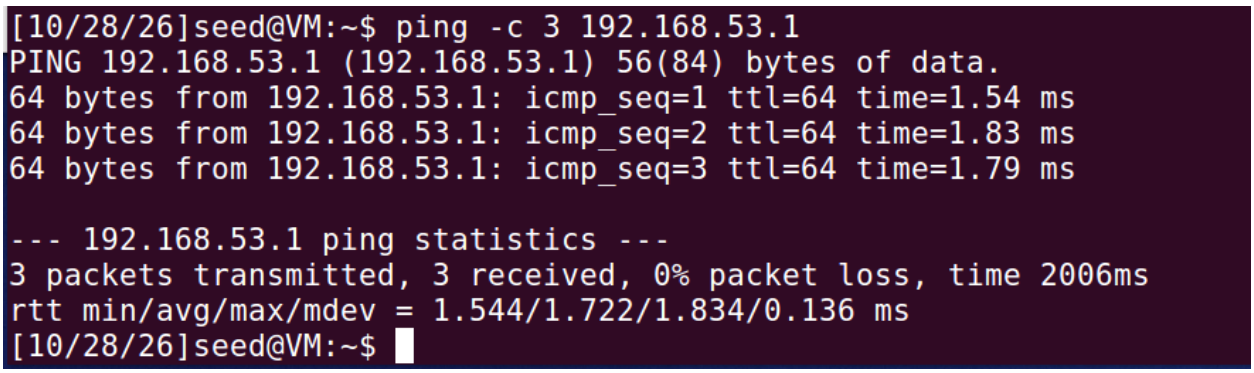
The client running



`ifconfig tun0` output



successful ping



Step 3: Set up routing

Routing tells a computer which way to send messages. On Host U, I added a route that sends everything for Facebook's range (57.144.138.0/24) into the tunnel. Without this route, the messages would go out the normal way and the firewall would block them.

```
sudo route add -net 57.144.138.0/24 tun0
route -n
```

On Host U

```
Terminal
[10/28/26]seed@VM:~$ sudo route add -net 57.144.138.0/24 tun0
[10/28/26]seed@VM:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.0.2.1        0.0.0.0         UG    100    0        0 enp0s3
10.0.2.0        0.0.0.0         255.255.255.0   U     100    0        0 enp0s3
57.144.138.0    0.0.0.0         255.255.255.0   U     0      0        0 tun0
169.254.0.0     0.0.0.0         255.255.0.0     U     1000   0        0 enp0s3
192.168.53.0    0.0.0.0         255.255.255.0   U     0      0        0 tun0
[10/28/26]seed@VM:~$
```

Gateway

```
Terminal
[10/28/26]seed@VM:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.0.2.1        0.0.0.0         UG    100    0        0 enp0s3
10.0.2.0        0.0.0.0         255.255.255.0   U     100    0        0 enp0s3
169.254.0.0     0.0.0.0         255.255.0.0     U     1000   0        0 enp0s8
192.168.53.0    0.0.0.0         255.255.255.0   U     0      0        0 tun0
192.168.60.0    0.0.0.0         255.255.255.0   U     100    0        0 enp0s8
[10/28/26]seed@VM:~$
```

The client needs an entry sending the blocked site's range to `tun0`. The server needs a route back to 192.168.53.0/24 via `tun0`, so replies can reach the client through the tunnel.

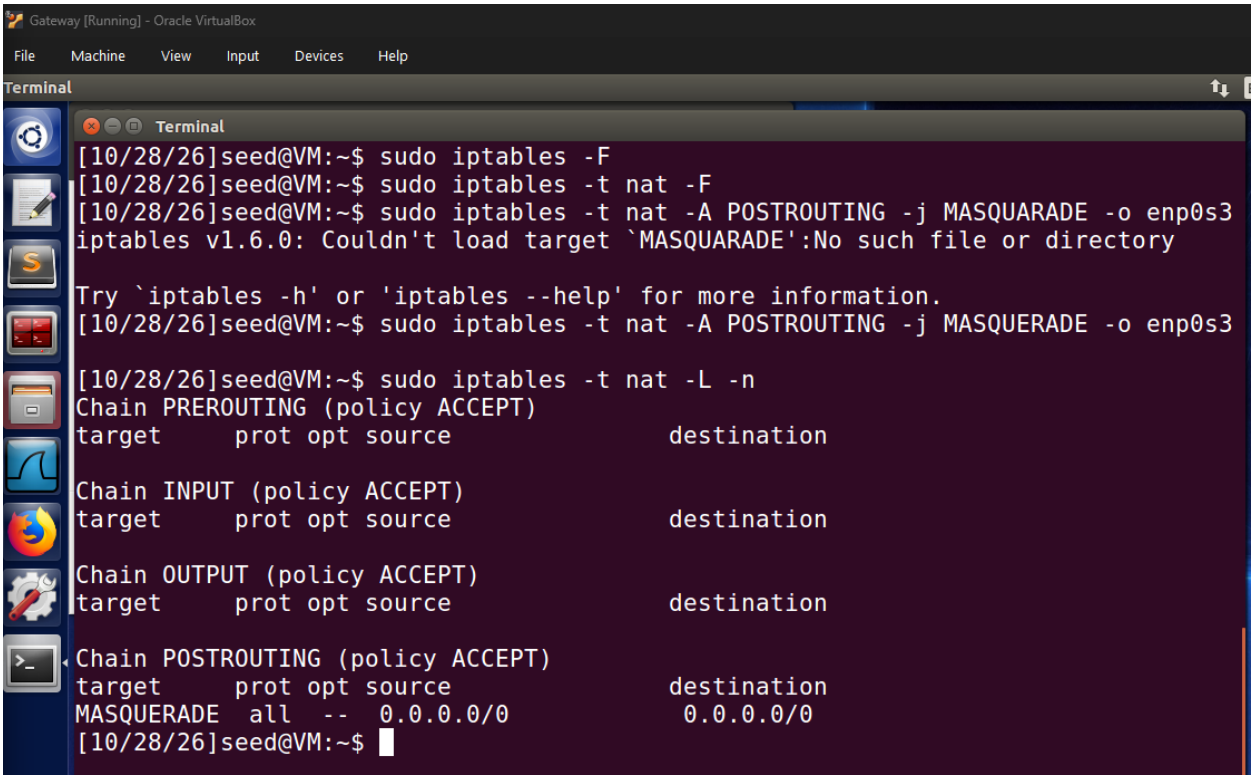
Step 4: Set up NAT (on the Gateway)

Host U's messages carry the tunnel address 192.168.53.5 as the sender. That address does not exist on the real network, so Facebook could not answer it. NAT fixes this. The Gateway changes the sender on each message to its own address. Facebook then answers the Gateway. This is why the answers come back to the Gateway first. The Gateway then sends them through the tunnel to Host U.

The first time I ran the NAT command, it showed an error. Running the same command again worked. The screenshot shows the rule is in place.

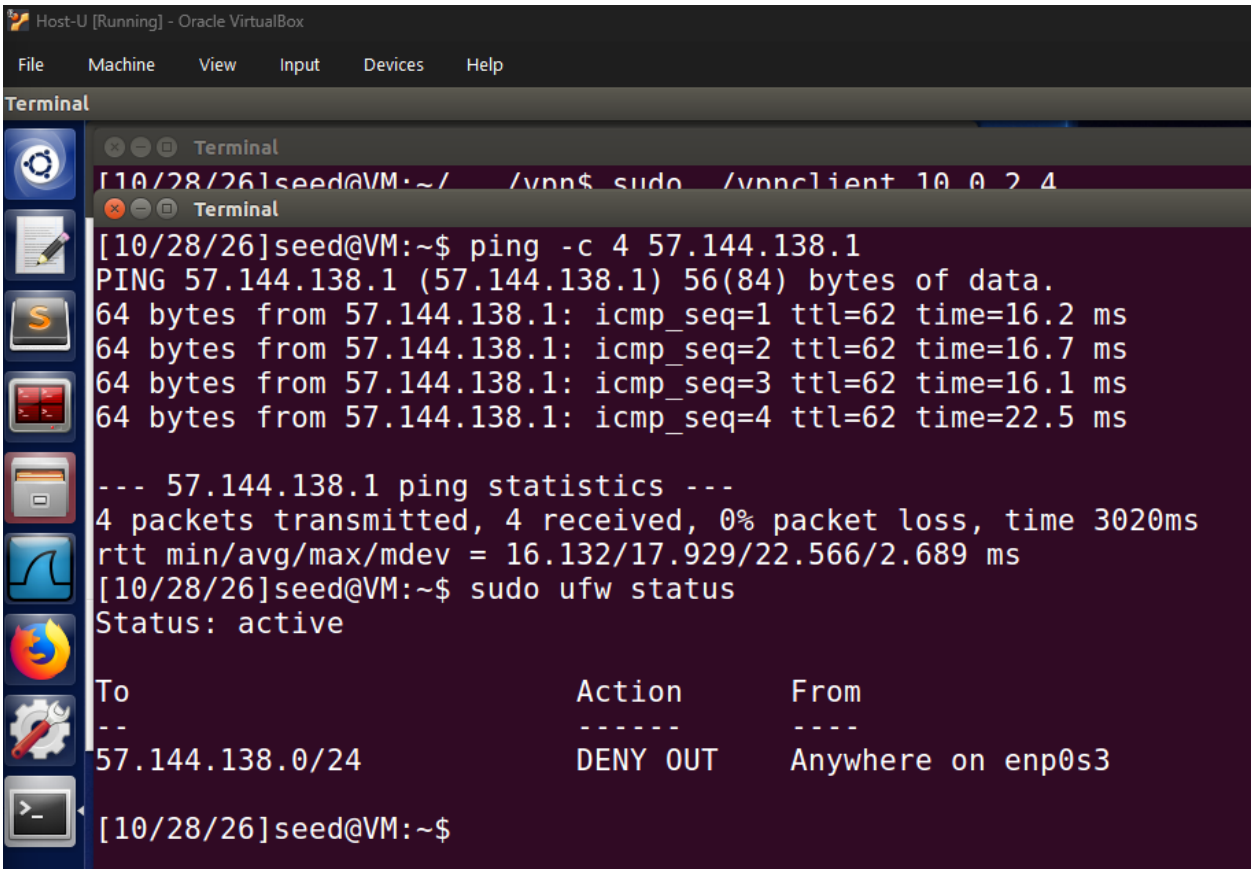
```
sudo iptables -F
sudo iptables -t nat -F
sudo iptables -t nat -A POSTROUTING -j MASQUERADE -o enp0s3
sudo iptables -t nat -L -n
```

MASQUERADE rule



Facebook ping and ufw status

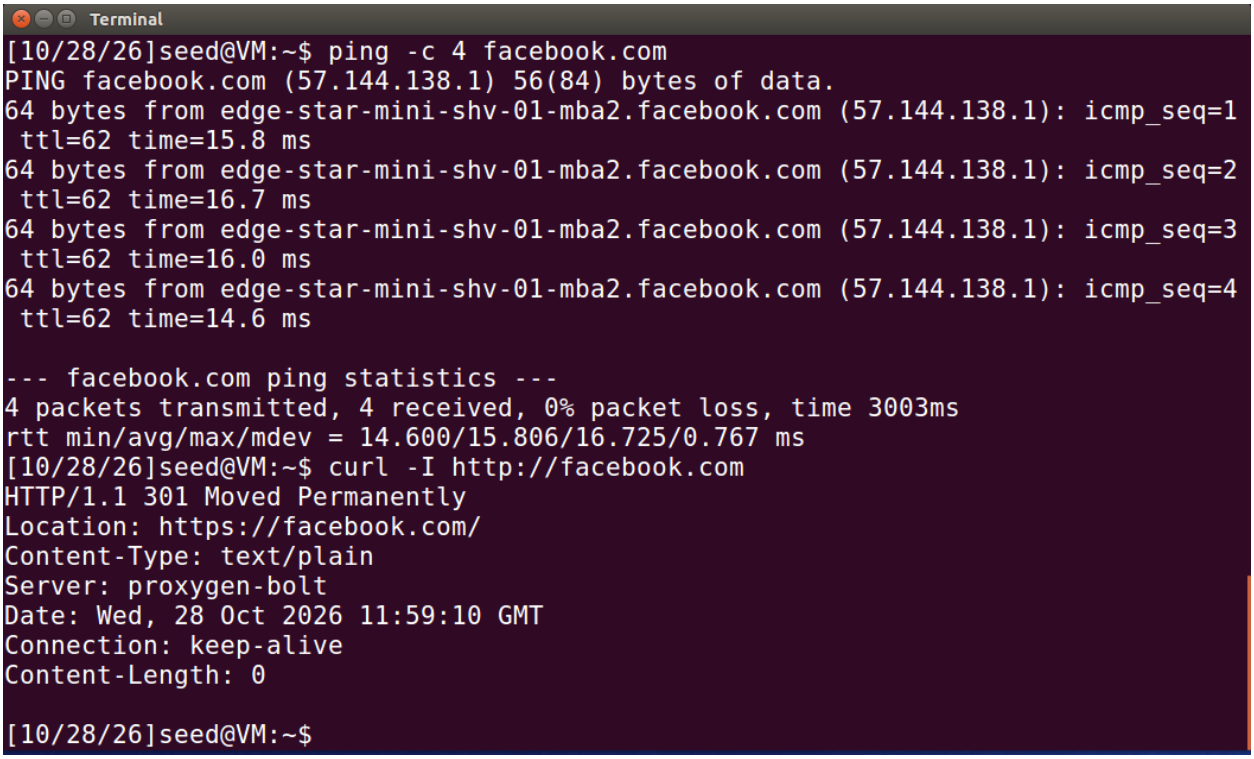
Host U can now reach Facebook's address, even with the firewall on. The firewall rule is still there in the list, so I did not turn it off or change it. All the pings were answered.



Web Traffic

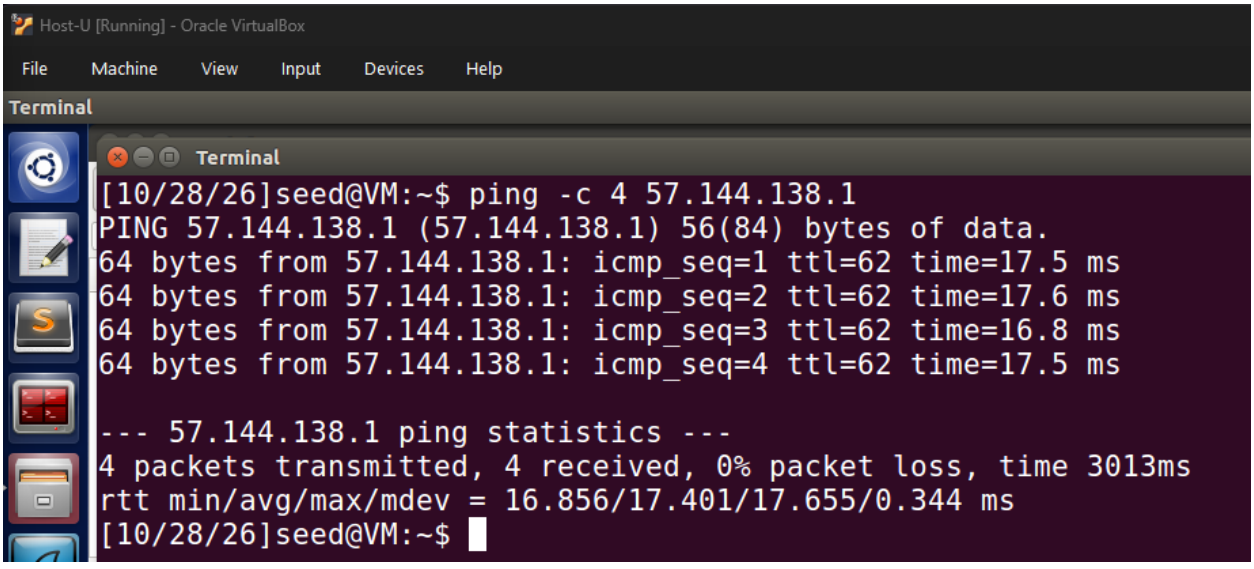
The tunnel also works for web traffic, not only pings. A web request to facebook.com got an answer from Facebook's server.

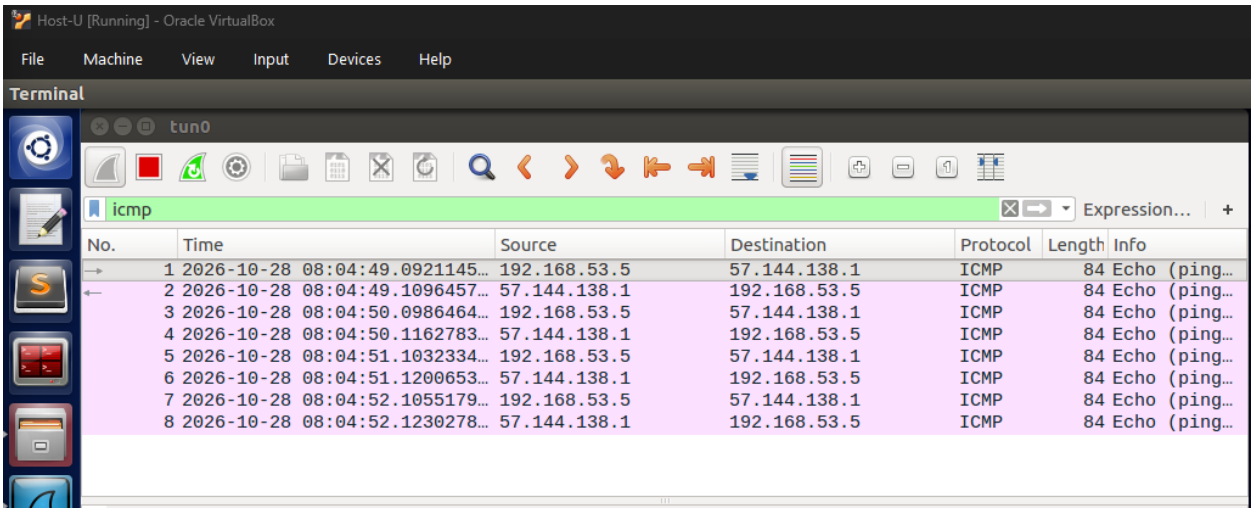
```
ping -c 4 facebook.com
curl -I http://facebook.com
```



Wireshark on Host U – ICMP filtered capture

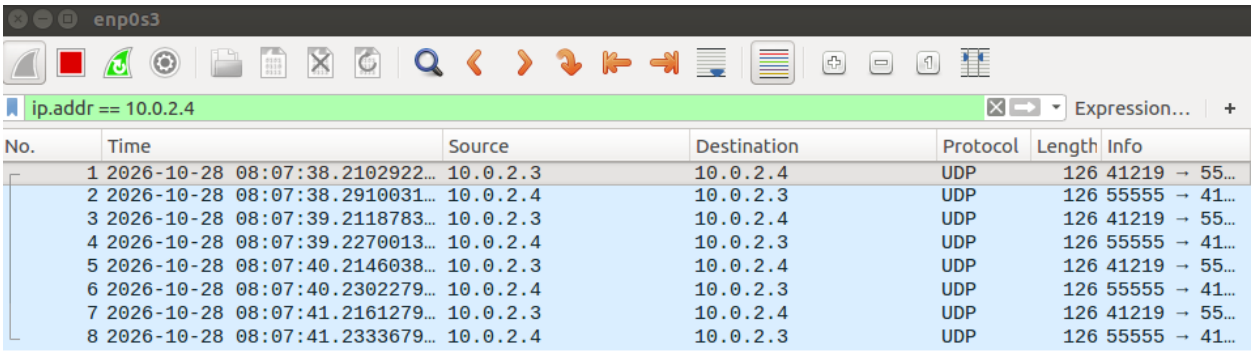
Wireshark is a tool that records network traffic. I used it to prove that the traffic went through the tunnel. This first capture is from the tunnel card (tun0) on Host U. It shows the original test messages between 192.168.53.5 (Host U) and 57.144.138.1 (Facebook). The requests go out and the answers come back.





ip.addr == 10.0.2.4 filtered capture

This second capture is from the real network card (enp0s3). It shows only UDP messages between Host U (10.0.2.3) and the Gateway (10.0.2.4). Nothing goes straight to Facebook. So the Facebook traffic is travelling inside these tunnel messages, and not through a back door.




```
▸ Frame 1: 126 bytes on wire (1008 bits), 126 bytes captured (1008 bits) on interface 0
▸ Ethernet II, Src: PcsCompu_35:a8:f6 (08:00:27:35:a8:f6), Dst: PcsCompu_6c:3b:10 (08:00:27:6c:3b:10)
▸ Internet Protocol Version 4, Src: 10.0.2.3, Dst: 10.0.2.4
▸ User Datagram Protocol, Src Port: 41219, Dst Port: 55555
▸ Data (84 bytes)
```

What this shows

I blocked Facebook on Host U, and it could not be reached. Then I built a tunnel to the Gateway and sent the blocked traffic through it. After that, Host U could reach Facebook again with the firewall still on. The captures show the original messages on the tunnel card, and only tunnel messages to the Gateway on the real network card.

One limit: this tunnel is not encrypted. Anyone who can capture the tunnel messages could read what is inside.